Nama: Muhammad Ilman Assiediqie
NIM: 12030123140343
Kelas: E

JAWABAN SOAL UTS

JAWABAN SOAL 1 — Desain Arsitektur Data, Normalisasi, dan Integritas Jurnal

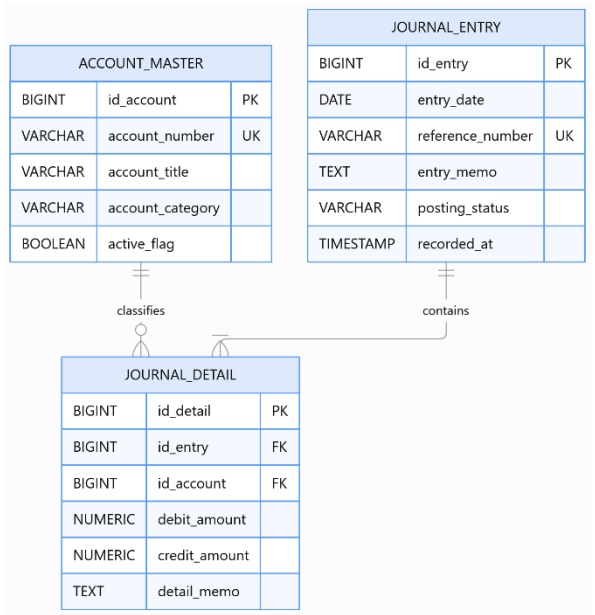
1.1 Perancangan Skema Nasional (ERD)

Sistem General Ledger memerlukan struktur basis data yang dapat menyimpan daftar akun, informasi umum transaksi, dan rincian pencatatan debit-kredit. Untuk itu, saya menggunakan tiga tabel yang memiliki fungsi berbeda, yaitu chart_of_accounts, journal_headers, dan journal_lines.

Tabel chart_of_accounts menyimpan daftar akun yang digunakan perusahaan. Tabel journal_headers berisi informasi utama transaksi, seperti tanggal, nomor referensi, deskripsi, dan status pencatatan. Sementara itu, journal_lines menyimpan rincian akun beserta nominal debit atau kredit untuk setiap transaksi.

Pemisahan tersebut memungkinkan satu transaksi memiliki beberapa baris jurnal tanpa harus mengulang informasi umum transaksi pada setiap baris.

Entity Relationship Diagram (ERD)



Penjelasan relasi:

1. chart_of_accounts ke journal_lines memiliki relasi satu-ke-banyak (one-to-many). Satu akun dapat digunakan dalam banyak baris jurnal, tetapi setiap baris jurnal hanya mengacu pada satu akun.
2. journal_headers ke journal_lines memiliki relasi satu-ke-banyak. Satu transaksi jurnal terdiri atas satu atau lebih baris jurnal, sedangkan setiap baris jurnal terhubung dengan satu header.
3. account_id dan header_id pada journal_lines bertindak sebagai foreign key yang menghubungkan rincian jurnal dengan tabel induknya.

1.2 DDL PostgreSQL dan Database Constraints

Struktur tabel dibuat menggunakan PostgreSQL. Setiap tabel mempunyai primary key sebagai identitas unik, sedangkan hubungan antartabel dijaga melalui foreign key. Selain itu, pembatasan nilai diterapkan pada kode akun, klasifikasi laporan, status posting, serta nominal debit dan kredit.

A. Membuat tabel chart_of_accounts
Tabel ini digunakan sebagai daftar akun atau Chart of Accounts (CoA). Kolom normal_balance menunjukkan posisi saldo normal akun, sedangkan report_classification menentukan kelompok laporan keuangan tempat akun disajikan.

```
CREATE TABLE chart_of_accounts (
    account_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    account_code VARCHAR(20) NOT NULL UNIQUE,
    account_name VARCHAR(100) NOT NULL,

    normal_balance VARCHAR(6) NOT NULL
        CHECK (normal_balance IN ('DEBIT', 'CREDIT')),

    report_classification VARCHAR(20) NOT NULL
        CHECK (
            report_classification IN (
                'ASSET',
                'LIABILITY',
                'EQUITY',
                'REVENUE',
                'EXPENSE'
            )
        ),

    is_active BOOLEAN NOT NULL DEFAULT TRUE
);
```

B. Membuat table journal_headers
Tabel ini menyimpan informasi utama setiap transaksi jurnal. Kolom reference_no dibuat unik agar satu nomor bukti tidak digunakan untuk mengidentifikasi lebih dari satu transaksi. Status posting dibatasi menjadi DRAFT atau POSTED

```
CREATE TABLE journal_headers (
    header_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    transaction_date DATE NOT NULL,
    reference_no VARCHAR(50) NOT NULL UNIQUE,
    description TEXT,

    posting_status VARCHAR(10) NOT NULL DEFAULT 'DRAFT'
        CHECK (posting_status IN ('DRAFT', 'POSTED')),

    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

C. Membuat table journal_lines

Tabel ini menyimpan rincian akun dan nilai setiap baris jurnal. Penggunaan ON DELETE RESTRICT mencegah header transaksi atau akun dihapus ketika masih direferensikan oleh baris jurnal. Check constraint memastikan nilai debit dan kredit tidak negatif serta tidak boleh sama-sama bernilai nol.

```
CREATE TABLE journal_lines (
    line_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

    header_id BIGINT NOT NULL,
    account_id BIGINT NOT NULL,

    debit NUMERIC(18,2) NOT NULL DEFAULT 0,
    credit NUMERIC(18,2) NOT NULL DEFAULT 0,

    CONSTRAINT fk_lines_header
        FOREIGN KEY (header_id)
        REFERENCES journal_headers(header_id)
        ON DELETE RESTRICT,

    CONSTRAINT fk_lines_account
        FOREIGN KEY (account_id)
        REFERENCES chart_of_accounts(account_id)
        ON DELETE RESTRICT,

    CONSTRAINT chk_debit_nonnegative
        CHECK (debit >= 0),

    CONSTRAINT chk_credit_nonnegative
        CHECK (credit >= 0),

    CONSTRAINT chk_journal_amount
```

```
    CHECK (debit > 0 OR credit > 0)
);
```

Constraint terakhir memastikan bahwa setiap baris jurnal mempunyai nilai pada sisi debit atau kredit. Namun, constraint tersebut belum melarang debit dan kredit bernilai positif secara bersamaan dalam satu baris. Apabila rancangan sistem mengharuskan setiap baris hanya mempunyai satu sisi nominal, constraint tambahan berikut dapat digunakan.

```
ALTER TABLE journal_lines
ADD CONSTRAINT chk_one_sided_entry
CHECK (
   (debit > 0 AND credit = 0)
   OR
   (credit > 0 AND debit = 0)
);
```

Catatan integritas: constraint di atas belum memastikan bahwa total debit sama dengan total kredit untuk seluruh baris dalam satu jurnal. Pemeriksaan tersebut harus dilakukan melalui fungsi atau prosedur posting di database, karena CHECK constraint PostgreSQL tidak dirancang untuk memvalidasi jumlah lintas beberapa baris.

1.3 Analisis Audit dan Keamanan Finansial

A. Alasan pemisahan tabel Header-Line

Pemisahan antara journal_headers dan journal_lines membuat data transaksi lebih terstruktur. Header menyimpan informasi umum yang berlaku untuk satu transaksi, sedangkan baris jurnal menyimpan akun dan nominal yang dapat berjumlah lebih dari satu.

Jika seluruh informasi ditempatkan dalam satu tabel, tanggal, nomor bukti, dan deskripsi transaksi harus diulang pada setiap baris jurnal. Pengulangan ini dapat meningkatkan risiko inkonsistensi, misalnya ketika deskripsi atau tanggal pada salah satu baris berbeda dari baris lainnya. Struktur Header-Line mengurangi risiko tersebut karena informasi umum transaksi disimpan pada satu tempat dan dihubungkan menggunakan header_id.

B. Hubungan dengan Bentuk Normal Ketiga (3NF)

Struktur yang dirancang mendukung penerapan Bentuk Normal Ketiga atau Third Normal Form (3NF), dengan ketentuan setiap tabel memenuhi bentuk normal sebelumnya dan atribut nonkunci bergantung pada kunci secara langsung, bukan melalui atribut nonkunci lainnya.

Pada chart_of_accounts, nama akun, saldo normal, dan klasifikasi laporan bergantung pada account_id. Pada journal_headers, tanggal transaksi, nomor referensi, deskripsi, dan status posting berkaitan dengan header_id. Sementara itu, pada journal_lines, nominal debit dan kredit berkaitan dengan identitas baris jurnal serta hubungan baris tersebut dengan header dan akun.

Dengan pemisahan ini, informasi akun tidak perlu diulang pada setiap transaksi, dan informasi header tidak perlu disalin ke setiap baris jurnal. Hal tersebut membantu mengurangi anomali penyisipan, perubahan, dan penghapusan data.

C. Penguatan Audit Trail dan integritas finansial

Struktur Header-Line memudahkan auditor menelusuri transaksi dari informasi utama pada journal_headers menuju rincian nominal dan akun pada journal_lines. Nomor referensi, tanggal transaksi, dan status posting juga membantu proses identifikasi serta pemeriksaan dokumen pendukung.

Meskipun demikian, pemisahan tabel saja belum cukup untuk menghasilkan audit trail yang lengkap. Sistem sebaiknya juga mencatat identitas pengguna, waktu perubahan, jenis tindakan, serta riwayat status transaksi. Setelah jurnal berstatus POSTED, perubahan atau penghapusan sebaiknya dilarang melalui mekanisme database, dan koreksi dilakukan menggunakan jurnal pembalik (reversing entry) agar riwayat transaksi sebelumnya tetap tersedia.

Selain itu, sistem perlu memvalidasi keseimbangan debit dan kredit sebelum jurnal diposting. Dengan demikian, validasi tidak hanya bergantung pada antarmuka pengguna (frontend), tetapi juga ditegakkan pada lapisan database sehingga transaksi yang dikirim langsung melalui API tetap harus memenuhi aturan akuntansi.

Kesimpulan: rancangan tiga tabel ini memisahkan data akun, informasi utama transaksi, dan rincian jurnal secara logis. Struktur tersebut mendukung normalisasi data, penelusuran transaksi, serta pengendalian integritas jurnal. Untuk penerapan yang lebih aman, rancangan perlu dilengkapi dengan validasi keseimbangan jurnal saat posting, pencatatan aktivitas pengguna, dan perlindungan terhadap perubahan jurnal yang telah diposting.

JAWABAN SOAL 2 — OTOMASI BISNIS, STORED PROCEDURES, DAN TRIGGER PERSEDIAAN

2.1 Otomatisasi Mutasi melalui Database Trigger

Pada sistem persediaan perpetual, perubahan kuantitas barang harus tercermin langsung dalam basis data setiap kali transaksi dicatat. Untuk memenuhi kebutuhan tersebut, digunakan trigger PostgreSQL yang memperbarui stock_qty berdasarkan jenis transaksi yang dimasukkan ke tabel inventory_transactions.

Dalam rancangan ini, transaksi PURCHASE menambah stok, SALE mengurangi stok, sedangkan ADJUSTMENT menyesuaikan stok berdasarkan kuantitas yang diberikan. Untuk ADJUSTMENT, kuantitas positif menambah stok dan kuantitas negatif mengurangi stok.

A. Fungsi trigger

```
CREATE OR REPLACE FUNCTION trg_update_stock_and_journal()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
```

```
    v_stock INT;
BEGIN
    IF NEW.qty = 0 THEN
        RAISE EXCEPTION
            'Kuantitas transaksi tidak boleh nol';
    END IF;

    IF NEW.trx_type IN ('PURCHASE', 'SALE')
      AND NEW.qty < 0 THEN
        RAISE EXCEPTION
            'Kuantitas PURCHASE dan SALE harus positif';
    END IF;

    SELECT stock_qty
    INTO v_stock
    FROM inventory_items
    WHERE item_id = NEW.item_id
    FOR UPDATE;

    IF NOT FOUND THEN
        RAISE EXCEPTION
            'Item dengan ID % tidak ditemukan',
            NEW.item_id;
    END IF;

    IF NEW.trx_type = 'PURCHASE' THEN

        UPDATE inventory_items
        SET stock_qty = stock_qty + NEW.qty
        WHERE item_id = NEW.item_id;
```

```
ELSIF NEW.trx_type = 'SALE' THEN

    IF v_stock < NEW.qty THEN
        RAISE EXCEPTION
            'Stok tidak mencukupi. Stok tersedia: %, jumlah penjualan: %',
            v_stock, NEW.qty;
    END IF;

    UPDATE inventory_items
    SET stock_qty = stock_qty - NEW.qty
    WHERE item_id = NEW.item_id;

ELSIF NEW.trx_type = 'ADJUSTMENT' THEN

    IF v_stock + NEW.qty < 0 THEN
        RAISE EXCEPTION
            'Penyesuaian ditolak karena menyebabkan stok negatif';
    END IF;

    UPDATE inventory_items
    SET stock_qty = stock_qty + NEW.qty
    WHERE item_id = NEW.item_id;

ELSE
    RAISE EXCEPTION
        'Jenis transaksi tidak dikenali: %',
        NEW.trx_type;
END IF;
```

```
    RETURN NEW;
END;
$$;
```

B. Pembuatan trigger

```
CREATE TRIGGER trg_inventory_stock
BEFORE INSERT ON inventory_transactions
FOR EACH ROW
EXECUTE FUNCTION trg_update_stock_and_journal();
```

Trigger menggunakan peristiwa BEFORE INSERT sehingga validasi dan pembaruan stok dilakukan sebelum baris transaksi dimasukkan. Apabila fungsi menghasilkan exception, perintah INSERT dibatalkan. Perubahan stok yang telah dilakukan dalam transaksi yang sama juga ikut dibatalkan apabila terjadi kegagalan.

Penggunaan SELECT ... FOR UPDATE berfungsi mengunci baris persediaan yang sedang diproses. Dengan demikian, transaksi lain yang hendak memperbarui item yang sama harus menunggu sampai transaksi yang memegang kunci selesai.

Implementasi ini mencakup pembaruan kuantitas dan pencegahan stok negatif. Meskipun nama fungsi memuat kata *journal*, fungsi tersebut belum membentuk jurnal akuntansi karena struktur tabel jurnal dan aturan pembentukan nilai nominalnya belum disediakan dalam soal 2.

Catatan implementasi: trigger di atas mengasumsikan setiap baris transaksi dimasukkan satu kali dan tidak diedit atau dihapus setelah pencatatan. Jika perubahan atau penghapusan transaksi persediaan diperbolehkan, sistem perlu menangani pembalikan mutasi stok secara terkontrol agar saldo tidak menjadi tidak konsisten.

2.2 Penanganan Concurrency dan Race Condition

Race condition dapat terjadi ketika dua kasir melakukan penjualan atas barang yang sama dalam waktu hampir bersamaan. Masalah ini menjadi serius ketika stok yang tersedia hanya satu unit.

Misalnya, stok barang BRG001 tersisa satu unit. Kasir pertama dan kasir kedua sama-sama mencoba menjual satu unit. Jika keduanya membaca saldo sebelum salah satu transaksi memperbaruinya, keduanya berpotensi menganggap barang masih tersedia. Tanpa pengamanan, sistem dapat menerima penjualan melebihi stok yang sebenarnya.

A. Menggunakan SELECT ... FOR UPDATE

Salah satu pendekatan adalah menggunakan penguncian baris agar hanya satu transaksi yang dapat memproses pembaruan stok suatu item pada satu waktu.

Contoh transaksi:

```
BEGIN;

SELECT stock_qty
FROM inventory_items
WHERE item_id = 'BRG001'
FOR UPDATE;

INSERT INTO inventory_transactions
  (item_id, trx_type, qty)
VALUES
  ('BRG001', 'SALE', 1);

COMMIT;
```

Ketika transaksi pertama mengunci baris BRG001, transaksi kedua yang mencoba memperoleh kunci pada baris yang sama harus menunggu. Setelah transaksi pertama selesai, transaksi kedua membaca kondisi stok terbaru melalui fungsi trigger. Jika stok sudah nol, penjualan kedua ditolak menggunakan RAISE EXCEPTION.

Dalam rancangan ini, penguncian dilakukan oleh trigger pada saat INSERT sehingga perlindungan tetap berlaku ketika transaksi dimasukkan melalui jalur aplikasi yang berbeda. Transaksi tetap perlu dijalankan menggunakan pengelolaan transaksi database yang benar.

B. Menggunakan isolasi Serializable

Alternatif lainnya adalah tingkat isolasi SERIALIZABLE.

```
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;

-- Lakukan pembacaan stok dan pencatatan penjualan
INSERT INTO inventory_transactions
  (item_id, trx_type, qty)
VALUES
  ('BRG001', 'SALE', 1);
```

COMMIT;

Pada tingkat isolasi ini, PostgreSQL mendeteksi konflik yang membuat transaksi tidak dapat diserialisasikan secara aman. Salah satu transaksi dapat dibatalkan dengan kesalahan serialization failure, sehingga aplikasi perlu menangani dan mencoba ulang transaksi yang gagal.

C. Constraint tambahan

Sebagai pengamanan tambahan, tabel persediaan dapat dilengkapi constraint berikut:

ALTER TABLE inventory_items

ADD CONSTRAINT chk_stock_non_negative

CHECK (stock_qty >= 0);

Constraint tersebut menjadi lapisan perlindungan tambahan agar saldo stok tidak tersimpan dalam keadaan negatif.

Dari kedua pendekatan tersebut, SELECT ... FOR UPDATE merupakan pilihan yang sesuai untuk kasus ketika transaksi bersamaan memperebutkan satu item persediaan. Sementara itu, SERIALIZABLE dapat digunakan apabila aplikasi membutuhkan jaminan serialisasi yang lebih luas dan memiliki mekanisme penanganan transaksi yang gagal. Keduanya tetap perlu didukung oleh validasi stok yang benar.

JAWABAN SOAL 3 — INTEGRASI PURWARUPA WEB FULL-STACK DAN ORKESTRASI AI TOOLS

3.1 Orkestrasi AI Tools

A. Gemini Canvas — Rancangan Formulir Entri Jurnal Kas Keluar

Purwarupa yang dirancang berupa formulir Voucher Entri Transaksi Kas Keluar. Formulir digunakan untuk memasukkan informasi transaksi dan rincian akun yang membentuk jurnal. Sistem harus menghitung jumlah debit dan kredit secara otomatis agar pengguna dapat mengetahui keseimbangan jurnal sebelum mengirimkan data.

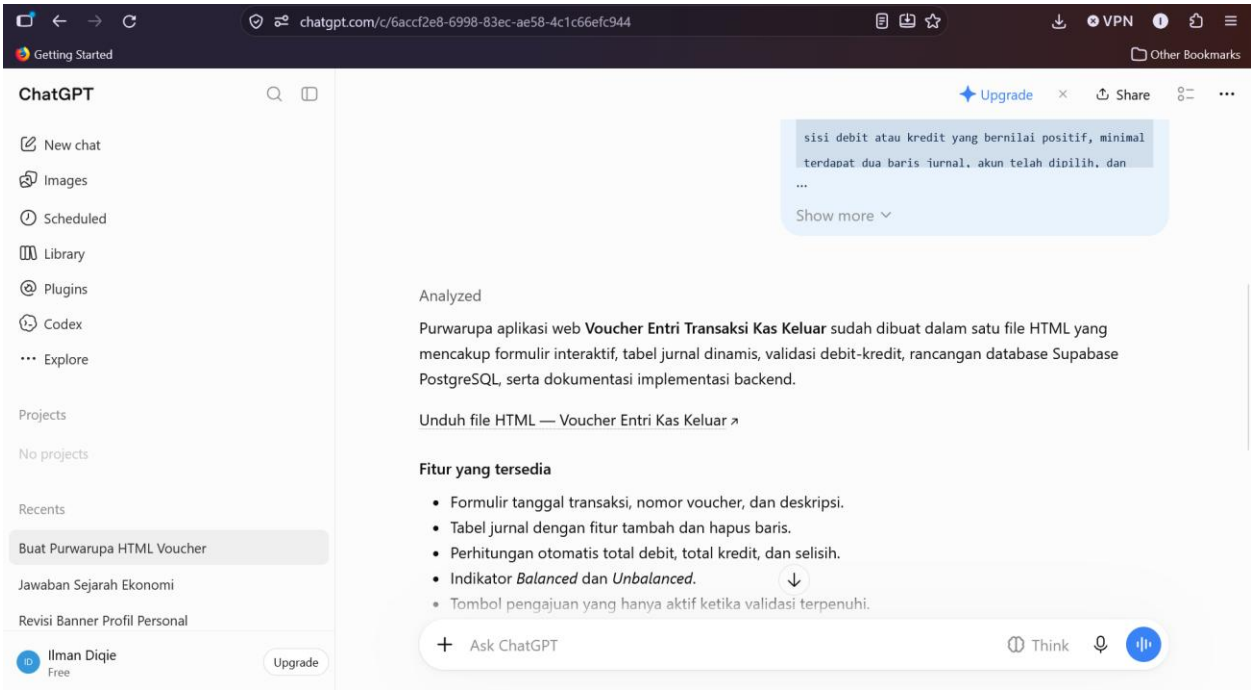
Komponen yang diperlukan dalam antarmuka meliputi:

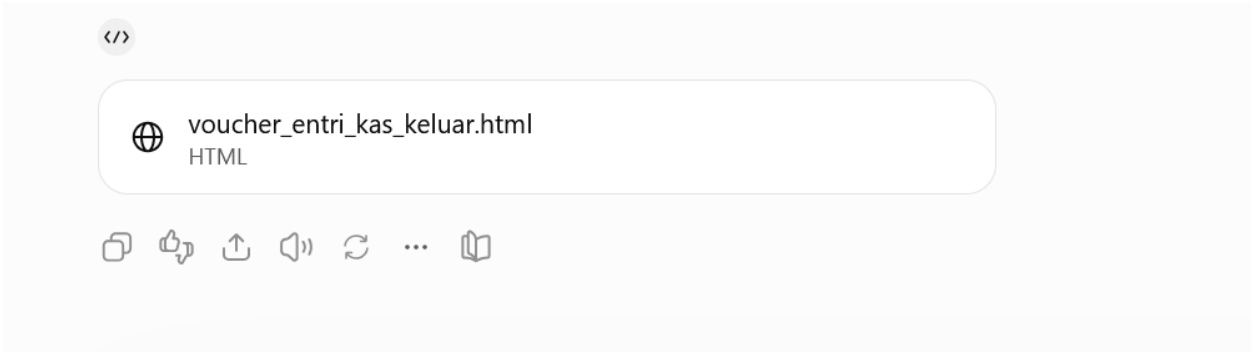
1. Kolom tanggal transaksi, nomor voucher, dan deskripsi.
2. Tabel jurnal dinamis dengan kolom akun, debit, dan kredit.
3. Fitur untuk menambahkan dan menghapus baris jurnal.
4. Perhitungan otomatis atas total debit, total kredit, dan selisihnya.
5. Indikator Balanced atau Unbalanced yang diperbarui ketika nominal berubah.

6. Tombol Submit yang hanya aktif ketika jurnal memenuhi ketentuan validasi.
7. Rancangan integrasi dengan Supabase/PostgreSQL untuk menyimpan transaksi.
8. Pencatatan identitas pembuat dan waktu pembuatan transaksi.

Validasi pada antarmuka digunakan untuk memberikan umpan balik langsung kepada pengguna. Akan tetapi, sistem tetap perlu melakukan validasi ulang di backend atau database karena pemeriksaan frontend dapat dilewati melalui permintaan API secara langsung.

Dokumentasi Prompt Gemini Canvas:



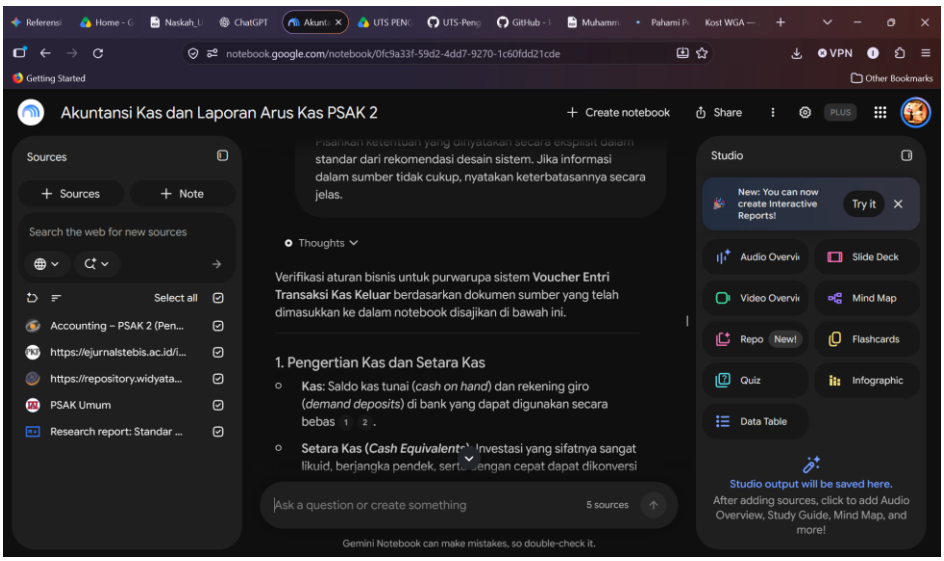


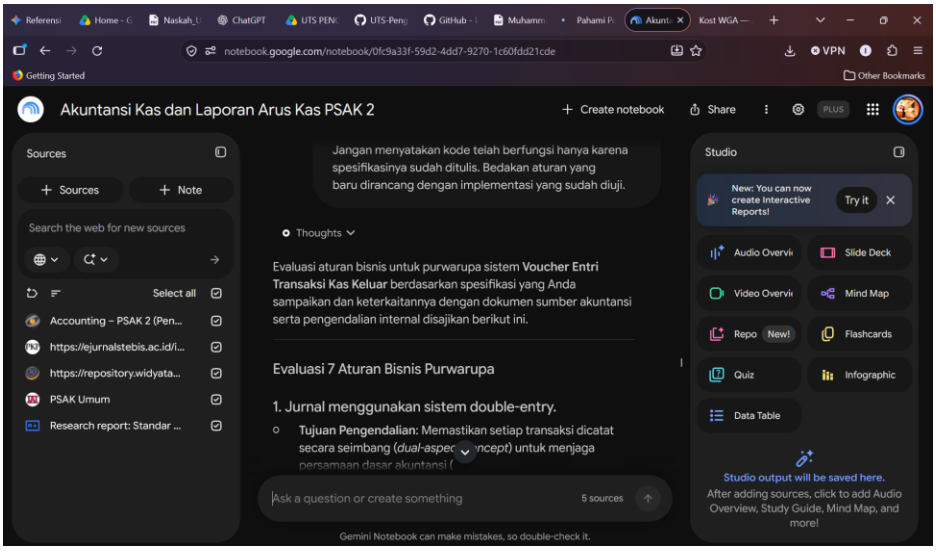
B. NotebookLM — Verifikasi Aturan Bisnis Berdasarkan PSAK

NotebookLM digunakan untuk membantu memeriksa apakah rancangan sistem pencatatan kas keluar telah sesuai dengan aturan bisnis dan standar akuntansi yang relevan. Pemeriksaan dilakukan berdasarkan dokumen yang dimasukkan sebagai sumber, bukan hanya berdasarkan jawaban umum AI.

Sumber yang sebaiknya disiapkan meliputi dokumen PSAK yang relevan mengenai penyajian laporan keuangan serta laporan arus kas, kebijakan pencatatan pengeluaran kas perusahaan jika tersedia, dan dokumentasi rancangan aplikasi beserta aturan validasinya. Gunakan dokumen resmi atau sumber yang dapat dipertanggungjawabkan dan periksa edisi standar yang berlaku.

Dokumentasi pada NotebookLM





Ringkasan hasil verifikasi NotebookLM

Berdasarkan hasil evaluasi yang diperoleh, aturan bisnis perlu diperiksa dari dua sisi. Dari sisi akuntansi, pemilihan akun dan arah debit-kredit harus mengikuti substansi transaksi. Pada transaksi pembayaran, akun yang didebit dapat berupa beban, pelunasan liabilitas, atau aset, sedangkan kas atau bank dikredit apabila terjadi pengeluaran dari akun tersebut. Penentuan akun lawan tetap bergantung pada transaksi yang sebenarnya.

Dari sisi pengendalian sistem, keseimbangan debit dan kredit merupakan persyaratan sebelum jurnal diposting. Namun, jurnal yang seimbang belum tentu benar secara akuntansi apabila akun yang digunakan atau pengakuan transaksinya tidak tepat. Oleh karena itu, pemeriksaan matematis harus disertai validasi akun dan aturan bisnis yang sesuai.

Perlindungan terhadap jurnal POSTED juga diperlukan untuk menjaga riwayat pencatatan. Apabila terjadi kesalahan, jurnal pembalik memungkinkan koreksi dilakukan tanpa menghilangkan catatan transaksi awal. Sementara itu, created_by, created_at, dan hash_checksum membantu penelusuran serta pemeriksaan integritas data, tetapi tidak menggantikan pengaturan hak akses dan pengujian keamanan database.

Catatan pengisian: paragraf ringkasan ini merupakan kerangka analisis yang perlu disesuaikan dengan hasil NotebookLM yang benar-benar muncul dari sumber yang kamu masukkan. Tambahkan referensi dokumen dan bagian pendukung berdasarkan jawaban aktualnya.

3.2 Mekanisme Jejak Audit (Audit Trail dan Immutability)

Audit trail diperlukan agar aktivitas pencatatan dapat ditelusuri, sedangkan immutability bertujuan mencegah perubahan langsung terhadap transaksi yang sudah final. Dalam rancangan ini, transaksi berstatus DRAFT masih dapat diperbaiki sesuai hak akses, tetapi transaksi POSTED harus dipertahankan dan dikoreksi melalui jurnal pembalik.

A. Spesifikasi fungsi backend

Fungsi backend dapat dirancang dengan Node.js/Express, Python/FastAPI, atau Supabase RPC. Tanggung jawab utamanya meliputi:

Fungsi	Tanggung Jawab
createVoucher()	Membuat voucher berstatus DRAFT serta mencatat identitas pembuat dan waktu pembuatan.
validateJournal()	Memeriksa akun, kelengkapan baris, nominal, dan keseimbangan debit-kredit.
postVoucher()	Memvalidasi jurnal, mengubah status menjadi POSTED, menghitung checksum, dan mencatat aktivitas posting.
preventPostedMutation()	Menolak perubahan atau penghapusan transaksi yang sudah diposting.
createReversingEntry()	Membuat jurnal pembalik yang terhubung dengan voucher asli.

writeAuditLog()	Menyimpan informasi aktivitas pengguna dan jenis tindakan yang dilakukan.
verifyChecksum()	Menghitung ulang checksum dan membandingkannya dengan nilai yang tersimpan.

Proses posting sebaiknya dilakukan secara atomik dalam satu transaksi database. Dengan demikian, perubahan status, validasi jurnal, pembentukan checksum, dan pencatatan log tidak terpisah sehingga berisiko menghasilkan data yang tidak konsisten.

B. Penambahan kolom audit dan tabel audit_logs

Kolom audit dapat ditambahkan pada tabel journal_headers yang telah dibuat pada Soal 1.

```
ALTER TABLE journal_headers

ADD COLUMN created_by UUID DEFAULT auth.uid(),

ADD COLUMN created_at TIMESTAMPTZ NOT NULL DEFAULT now(),

ADD COLUMN hash_checksum CHAR(64),

ADD COLUMN reversal_of BIGINT

  REFERENCES journal_headers(header_id)

  ON DELETE RESTRICT;
```

Pada implementasi Supabase, auth.uid() dapat digunakan untuk memperoleh identitas pengguna yang terautentikasi sesuai konteks permintaan. Backend harus memastikan identitas tersebut benar-benar berasal dari sesi yang valid, bukan dari nilai yang bebas dikirim oleh browser. Jika menggunakan PostgreSQL di luar Supabase, mekanisme pengisian identitas perlu disesuaikan.

Tabel untuk menyimpan riwayat aktivitas dapat dibuat sebagai berikut.

```
CREATE TABLE audit_logs (

  log_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```

```
    header_id BIGINT NOT NULL
        REFERENCES journal_headers(header_id)
        ON DELETE RESTRICT,
    action VARCHAR(30) NOT NULL,
    performed_by UUID,
    performed_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    details TEXT
);
CREATE TABLE audit_logs (
    log_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    header_id BIGINT NOT NULL
        REFERENCES journal_headers(header_id)
        ON DELETE RESTRICT,
    action VARCHAR(30) NOT NULL,
    performed_by UUID,
    performed_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    details TEXT
);
```

Tabel tersebut dapat mencatat tindakan seperti CREATE, POST, dan REVERSE. Untuk menjadikannya audit trail yang andal, pencatatan perlu dilakukan secara konsisten oleh database atau backend dan akses untuk mengubah maupun menghapus log harus dibatasi.

C. Mencegah perubahan pada header yang sudah POSTED

Berikut contoh trigger PostgreSQL untuk menolak UPDATE dan DELETE terhadap header yang sudah diposting.

```
CREATE OR REPLACE FUNCTION
public.prevent_posted_header_mutation()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
   IF OLD.posting_status = 'POSTED' THEN
      RAISE EXCEPTION
         'Voucher % sudah POSTED dan tidak dapat diubah atau dihapus',
         OLD.reference_no;
   END IF;

   IF TG_OP = 'DELETE' THEN
      RETURN OLD;
   END IF;

   RETURN NEW;
END;
$$;

CREATE TRIGGER trg_lock_posted_headers
BEFORE UPDATE OR DELETE ON journal_headers
FOR EACH ROW
EXECUTE FUNCTION public.prevent_posted_header_mutation();
```

Fungsi tersebut memeriksa status lama sebelum mengizinkan perubahan. Dengan demikian, header yang telah berstatus POSTED tidak dapat diubah atau dihapus melalui operasi tersebut. Sementara itu, perubahan status dari DRAFT menjadi POSTED tetap dimungkinkan karena status lama masih DRAFT. Validasi khusus tetap diperlukan sebelum perubahan status tersebut dilakukan.

D. Mencegah perubahan pada baris jurnal

Perlindungan tidak cukup diterapkan pada header karena rincian transaksi berada pada tabel journal_lines. Oleh karena itu, setiap operasi INSERT, UPDATE, atau DELETE pada baris jurnal harus memeriksa status header induknya.

```
CREATE OR REPLACE FUNCTION
public.prevent_posted_line_mutation()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_header_id BIGINT;
    v_status TEXT;
BEGIN
    IF TG_OP = 'INSERT' THEN
        v_header_id := NEW.header_id;
    ELSE
        v_header_id := OLD.header_id;
    END IF;

    SELECT posting_status
    INTO v_status
    FROM journal_headers
    WHERE header_id = v_header_id
    FOR UPDATE;

    IF NOT FOUND THEN
```

```
        RAISE EXCEPTION
            'Header jurnal tidak ditemukan';
    END IF;

    IF v_status = 'POSTED' THEN
        RAISE EXCEPTION
            'Baris jurnal milik voucher POSTED tidak dapat diubah';
    END IF;

    IF TG_OP = 'DELETE' THEN
        RETURN OLD;
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_lock_posted_lines
BEFORE INSERT OR UPDATE OR DELETE ON journal_lines
FOR EACH ROW
EXECUTE FUNCTION public.prevent_posted_line_mutation();

CREATE OR REPLACE FUNCTION
public.prevent_posted_line_mutation()
RETURNS TRIGGER
LANGUAGE plpgsql
```

```
AS $$
DECLARE
    v_header_id BIGINT;
    v_status TEXT;
BEGIN
    IF TG_OP = 'INSERT' THEN
        v_header_id := NEW.header_id;
    ELSE
        v_header_id := OLD.header_id;
    END IF;

    SELECT posting_status
    INTO v_status
    FROM journal_headers
    WHERE header_id = v_header_id
    FOR UPDATE;

    IF NOT FOUND THEN
        RAISE EXCEPTION
            'Header jurnal tidak ditemukan';
    END IF;

    IF v_status = 'POSTED' THEN
        RAISE EXCEPTION
            'Baris jurnal milik voucher POSTED tidak dapat diubah';
    END IF;
```

```
    IF TG_OP = 'DELETE' THEN
        RETURN OLD;
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_lock_posted_lines
BEFORE INSERT OR UPDATE OR DELETE ON journal_lines
FOR EACH ROW
EXECUTE FUNCTION public.prevent_posted_line_mutation();
```

Fungsi tersebut memeriksa header yang terkait dengan baris yang sedang diproses. Penguncian baris header membantu mengoordinasikan operasi terhadap header dan rinciannya.

Untuk penerapan produksi, operasi UPDATE yang memindahkan suatu baris ke header lain juga harus memeriksa header asal dan header tujuan. Mekanisme posting harus menggunakan penguncian yang konsisten agar tidak terjadi perubahan baris secara bersamaan ketika status header berubah menjadi POSTED. Selain itu, hak akses langsung ke tabel perlu dibatasi agar pengguna hanya dapat melakukan perubahan melalui jalur yang diizinkan.

E. Implementasi created_by, created_at, dan hash_checksum

Ketiga field tersebut memiliki fungsi yang berbeda.

1. created_by

Kolom ini menyimpan identitas pengguna yang membuat voucher. Informasi tersebut digunakan untuk mengetahui pihak yang bertanggung jawab atas pembuatan transaksi.

Nilainya harus berasal dari identitas yang diverifikasi oleh sistem, bukan hanya dari input formulir.

2. created_at

Kolom ini mencatat waktu pembuatan voucher menggunakan waktu server database. Penggunaan DEFAULT now() membantu memastikan bahwa waktu dicatat secara konsisten tanpa bergantung pada jam perangkat pengguna.

3. hash_checksum

Kolom ini menyimpan nilai hash yang dihasilkan dari representasi data transaksi. Data yang dihitung dapat mencakup nomor voucher, tanggal, deskripsi, kode akun, dan nilai debit-kredit. Format data harus disusun secara konsisten agar perhitungan ulang menghasilkan nilai yang sama apabila data tidak berubah.

Secara konseptual, perhitungannya dapat ditulis:

hash_checksum = SHA256(canonical_voucher_data)

hash_checksum = SHA256(canonical_voucher_data)

Ketika verifikasi dilakukan, sistem menghitung ulang hash berdasarkan data transaksi yang tersimpan dan membandingkannya dengan checksum sebelumnya. Perbedaan nilai menjadi indikasi bahwa data yang diperiksa mungkin telah berubah.

Namun, checksum biasa tidak menjamin keaslian data apabila pihak yang tidak berwenang dapat mengubah data dan checksum secara bersamaan. Karena itu, mekanisme tersebut perlu didukung pembatasan hak akses, perlindungan log, dan pengelolaan kunci yang aman apabila menggunakan HMAC atau tanda tangan digital.

F. Validasi dan proses posting jurnal

Sebelum jurnal berstatus POSTED, sistem harus memeriksa sekurang-kurangnya dua baris jurnal yang valid, akun yang tersedia, nominal yang tidak negatif, serta kesamaan total debit dan kredit. Pemeriksaan harus dilakukan pada backend atau database, bukan hanya pada antarmuka.

Contoh fungsi validasi berikut dapat digunakan sebagai bagian dari mekanisme posting.

CREATE OR REPLACE FUNCTION public.post_voucher(

```
    p_header_id BIGINT
)
RETURNS VOID
LANGUAGE plpgsql
AS $$
DECLARE
    v_status TEXT;
    v_count INT;
    v_debit NUMERIC(18,2);
    v_credit NUMERIC(18,2);
BEGIN
    SELECT posting_status
    INTO v_status
    FROM journal_headers
    WHERE header_id = p_header_id
    FOR UPDATE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Voucher tidak ditemukan';
    END IF;

    IF v_status <> 'DRAFT' THEN
        RAISE EXCEPTION
            'Hanya voucher DRAFT yang dapat diposting';
    END IF;
```

```
SELECT
    COUNT(*),
    COALESCE(SUM(debit), 0),
    COALESCE(SUM(credit), 0)
INTO v_count, v_debit, v_credit
FROM journal_lines
WHERE header_id = p_header_id;

IF v_count < 2
   OR v_debit <= 0
   OR v_credit <= 0
   OR v_debit <> v_credit THEN
    RAISE EXCEPTION
        'Jurnal tidak valid: jumlah baris %, debit %, kredit %',
        v_count, v_debit, v_credit;
END IF;

-- Pembuatan checksum dan pencatatan audit
-- dilakukan pada bagian lanjutan proses posting.

UPDATE journal_headers
SET posting_status = 'POSTED'
WHERE header_id = p_header_id;

INSERT INTO audit_logs (
    header_id,
```

```
        action,
        performed_by,
        details
    )
    VALUES (
        p_header_id,
        'POST',
        auth.uid(),
        'Voucher berhasil diposting setelah validasi'
    );
END;
$$;
```

G. Mekanisme jurnal pembalik (Reversing Entry/Storno)

Kesalahan pada jurnal yang telah diposting tidak diperbaiki dengan mengubah atau menghapus transaksi aslinya. Sistem membuat jurnal baru yang merujuk pada voucher sebelumnya dan membalik sisi debit serta kreditnya.

Hubungan antara voucher awal dan voucher pembalik disimpan menggunakan kolom reversal_of. Dengan demikian, auditor dapat memeriksa transaksi awal, alasan koreksi, serta jurnal pembalik yang dibuat.

Contoh berikut menggambarkan pembuatan voucher pembalik berstatus DRAFT.

```
CREATE OR REPLACE FUNCTION public.create_reversing_entry(
    p_header_id BIGINT,
    p_new_ref VARCHAR,
    p_reason TEXT
)
```

```
RETURNS BIGINT
LANGUAGE plpgsql
AS $$
DECLARE
    v_new_id BIGINT;
BEGIN
    IF NOT EXISTS (
        SELECT 1
        FROM journal_headers
        WHERE header_id = p_header_id
          AND posting_status = 'POSTED'
    ) THEN
        RAISE EXCEPTION
            'Hanya voucher POSTED yang dapat dibalik';
    END IF;

    INSERT INTO journal_headers (
        transaction_date,
        reference_no,
        description,
        posting_status,
        reversal_of
```

```
)
SELECT
    CURRENT_DATE,
    p_new_ref,
    'Jurnal pembalik atas ' || reference_no
        || ': ' || p_reason,
    'DRAFT',
    header_id
FROM journal_headers
WHERE header_id = p_header_id
RETURNING header_id INTO v_new_id;

INSERT INTO journal_lines (
    header_id,
    account_id,
    debit,
    credit
)
SELECT
    v_new_id,
    account_id,
    credit,
```

```
        debit
    FROM journal_lines
    WHERE header_id = p_header_id;

    INSERT INTO audit_logs (
        header_id,
        action,
        performed_by,
        details
    )
    VALUES (
        v_new_id,
        'REVERSE',
        auth.uid(),
        'Jurnal pembalik dibuat untuk voucher '
            || p_header_id || ': ' || p_reason
    );

    RETURN v_new_id;
END;
$$;
```

Fungsi tersebut hanya membuat jurnal pembalik dalam status DRAFT. Voucher pembalik harus melewati validasi dan proses posting tersendiri sebelum menjadi POSTED. Untuk penerapan yang aman, pemeriksaan terhadap voucher asal harus disertai penguncian transaksi dan aturan yang mencegah pembuatan pembalik ganda secara tidak sengaja.

H. Kesimpulan

Rancangan audit trail dan immutability bertujuan menjaga agar transaksi yang telah diposting tetap dapat ditelusuri dan tidak diubah melalui operasi normal aplikasi. Informasi created_by dan created_at membantu mengidentifikasi pembuat serta waktu transaksi, sedangkan hash_checksum mendukung pemeriksaan integritas data.

Jika ditemukan kesalahan, jurnal pembalik menyediakan mekanisme koreksi tanpa menghilangkan catatan awal. Namun, keberhasilan rancangan ini tetap bergantung pada penerapan trigger, hak akses, validasi posting, penguncian transaksi, serta pengujian pada database yang digunakan.